

Ntole

Ride-hailing platform for Malawi. One Next.js application serving riders, drivers, and platform operators from a shared Supabase backend.

AUDIENCE

Engineering team

CODEBASE

Lordsi/Ntole · main

GENERATED

21 May 2026

SURFACES

Rider · Driver · Admin

Table of contents

1. What Ntole is	9. Admin surface
2. Technology stack	10. Backend API endpoints
3. Repository layout	11. Shared libraries (<code>lib/</code>)
4. Authentication & access control	12. Realtime, maps & geolocation
5. Database schema	13. Payments
6. Shared & public pages	14. Environment configuration
7. Rider surface	15. Development & deployment workflow
8. Driver surface	16. Glossary

1 • What Ntole is

Ntole is a ride-hailing progressive web app targeted at Malawi. A single Next.js 15 application ships three role-scoped surfaces from one codebase, all sharing a Supabase backend:

- **Rider** — the public-facing hailing experience: enter pickup and destination, see real-time fare quotes per ride tier, request a ride, track the assigned driver on a live map, and pay.
- **Driver** — onboarding/compliance wizard, online/offline toggle, incoming ride requests, in-trip state machine (arrive → start → end), and earnings.
- **Admin** — operational console: users, drivers (including review of submitted applications), rides, dynamic pricing tiers, and a fleet live-map.

The fare model is the standard Uber-style equation, applied per tier:

```
fare = max(min_fare, (base + per_km × distance + per_min × duration) × surge)
```

Tiers, rates, and surge multipliers are stored in `ride_tiers` and are admin-editable.

2 • Technology stack

Framework	Next.js 15 (App Router) + React 19, written in TypeScript.
Styling	Tailwind CSS with a Material-3-inspired token system (custom <code>tailwind.config.ts</code>). Light + dark themes via a <code>ThemeProvider</code> .
Typography	Plus Jakarta Sans (single sans family across display, body, and labels
Backend	Supabase — Postgres for data, GoTrue for auth, Realtime for live channels, Storage for driver document and avatar uploads.
Authorization	Row-Level Security policies enforce that riders only see their own rides, drivers only see their own driver record, etc. Admins have a <code>SECURITY DEFINER</code> escape hatch via <code>is_admin()</code> .
Maps	<code>react-leaflet</code> + CartoDB tiles (dark and Voyager light); OSRM (<code>lib/maps/osrm.ts</code>) for route + ETA; Nominatim

(`lib/maps/nominatim.ts`) for forward + reverse geocoding via internal proxy routes.

Payments

Provider-abstracted: `PaymentProvider` interface with Stripe and a mock implementation selected by `PAYMENT_PROVIDER`.

PWA

Service worker registration + install prompt for "Add to home screen" Manifest in `public/manifest.webmanifest`.

Hosting

Render.com (auto-deploys from the `main` branch).

Testing

Vitest (currently exercising the pricing engine — see `lib/pricing/fare.test.ts`).

3 • Repository layout

<code>app/</code>	Next.js routes (App Router)
<code> page.tsx</code>	Smart entry – redirects by role
<code> layout.tsx</code>	Global <code><html></code> , fonts, <code>ThemeProvider</code> , PWA bootstrap
<code> globals.css</code>	Tailwind layer + design-system tokens
<code> (auth)/login/</code>	Sign in / sign up
<code> auth/callback/</code>	Email-link / OAuth code exchange
<code> rider/</code>	Rider surface
<code> driver/</code>	Driver surface
<code> admin/</code>	Admin surface
<code> api/</code>	Server route handlers
<code>components/</code>	React components, grouped by surface
<code> rider/ driver/ admin/ shared/ ui/ map/ chat/</code>	
<code>lib/</code>	Server & client utilities, no JSX
<code> auth/</code>	Routing helpers + session guards
<code> supabase/</code>	Browser, server, middleware clients + types
<code> maps/</code>	OSRM, Nominatim, geolocation hook
<code> pricing/</code>	Fare engine (+ unit tests)
<code> payments/</code>	<code>PaymentProvider</code> interface + Stripe / mock
<code> realtime/</code>	Channel helpers (driver location, ride updates)
<code> notifications/</code>	Builders for the notifications sheet
<code> admin/</code>	Moderation helpers + signed storage URLs
<code> driver/</code>	Driver application schema
<code>supabase/</code>	
<code> migrations/</code>	<code>0001_init.sql</code> , <code>0002_profile_extensions.sql</code> , <code>0003_driver_compliance</code>
<code> seed.sql</code>	Demo data (tiers + seeded users)
<code>middleware.ts</code>	Auth gate + role guard
<code>public/</code>	PWA icons, manifest, static assets

4 · Authentication & access control

Sign-in flow

- Email + password via Supabase Auth (`signInWithPassword` / `signUp`).
- Confirmation emails (when enabled in Supabase) come back through `app/auth/callback/route.ts`, which exchanges the code for a session cookie.
- Three seeded accounts ship in `supabase/seed.sql` for QA (rider, driver, admin).

Middleware

`middleware.ts` runs on every request and:

1. Refreshes the Supabase session cookie (`updateSession`).
2. Lets public paths through: `/`, `/login`, `/rider`, and anything under `/auth/`.
3. Otherwise requires a session — redirects to `/login?next=...` if absent.
4. Reads the user's role from `profiles` and gates top-level segments via `canAccess()` in `lib/auth/routing.ts`.
5. Forces unapproved drivers through `/driver/apply` before they can reach the rest of the driver surface.

Server components

Protected server components call `requireRole(...roles)` (in `lib/auth/session.ts`) which redirects mismatched users to their own home.

Row Level Security

Every table has RLS enabled. Representative policies:

- `profiles_self_read` — a user reads their own row; admins read all.
- `profiles_ride_partner_read` — riders and drivers can read each other's basic profile, only for rides they share.
- `rides_participant_read / update` — only the rider, the driver, or an admin.
- `messages_participant_*` — chat messages are visible only to the two ride participants.
- `favorite_drivers_self_*` — riders can favourite drivers, only manage their own list.
- `"avatars self write"` on `storage.objects` — users upload only into `avatars/<uid>/...`.

Atomic ride acceptance uses a `SECURITY DEFINER` RPC (`accept_ride(ride)`) so two drivers can't both win the same request.

5 · Database schema

Three migrations under `supabase/migrations/`. All idempotent.

Core tables (migration 0001)

Table	What it stores
profiles	One row per <code>auth.users</code> row. <code>role</code> (rider / driver / admin), <code>full_name</code> , <code>phone</code> , <code>avatar_url</code> , <code>rating</code> , <code>trip_count</code> , <code>safety_rating</code> .
ride_tiers	Pricing tiers (Standard, Comfort, Lux, etc.). Base fare, per-km, per-minute, min fare, currency, surge multiplier, seat count, ordering, active flag.
vehicles	Driver-owned vehicles. Make, model, plate, color, seats, tier.
drivers	Driver-specific data. Linked vehicle, status (offline / online / on_trip), current GPS coords, last-seen timestamp, license number, verification flag.
rides	The lifecycle of a ride. Foreign keys to rider / driver / vehicle / tier. Status enum, pickup & drop coords + address, quoted vs actual distance & duration, fare, currency, surge multiplier, and a full set of timestamps (requested, accepted, arrived, started, completed, cancelled).
messages	In-ride chat between rider and driver.
payments	One row per payment intent. Provider, provider intent id, amount, status (pending / authorized / paid / failed / refunded).
ratings	One-per-ride-per-rater 1–5 star rating + optional comment. A trigger recomputes the recipient's <code>profiles.rating</code> average.
app_config	Single-row global settings (default currency, match radius, ping interval, surge).

Profile extensions (migration 0002)

- `profiles.home_lat` / `home_lng` / `home_address` — saved "Home" pickup pin used by the rider quick-chip rail.
- `favorite_drivers` (`rider_id`, `driver_id`) — riders can star drivers they've ridden with.
- `storage.buckets.avatars` — public read, write only into `<uid>/...`. Powers self-served profile picture uploads.

Driver compliance (migration 0003)

Adds onboarding, moderation, and complaint primitives:

- `drivers` gains `approval_status` (draft → submitted → approved / rejected / banned), normalized national ID + license fields, document paths in Storage, requested vs admin-assigned tier, rating overrides, warning count, ban metadata.
- `banned_identifiers` — reuse-resistant ban registry keyed by national ID + license.
- `complaints` — rider/driver complaints with category and status.
- `driver_moderation_log` — audit trail of every approve / reject / warn / ban / unban / tier change / rating override / verify action.

Key Postgres helpers

- `haversine_km(...)` — great-circle distance in km.
- `find_nearby_drivers(origin_lat, origin_lng, tier, radius, limit)` — returns the N closest online drivers in a tier within a radius.
- `accept_ride(ride)` — atomic claim used by `PATCH /api/rides/[id]`.
- `is_admin()` — predicate used by every admin-gated RLS policy.
- Triggers: `touch_updated_at` on all mutable tables, `handle_new_user` seeds a profile when a Supabase auth user is inserted, `recompute_rating` updates running averages, `increment_trip_counts` on completion.

6 • Shared & public pages

`/` • Smart entry

`app/page.tsx`

Reads the session and redirects: anonymous → `/rider` (public preview), rider → `/rider`, driver → `/driver`, admin → `/admin`.

`/login` • Sign in / sign up

`app/(auth)/login/page.tsx`

Toggle between **Sign in** and **Create account**. Restores any persisted pending-ride form via `sessionStorage` after success. Lists seeded demo accounts.

`/auth/callback` • Email link handler

`app/auth/callback/route.ts`

Exchanges Supabase's `code` for a session cookie, then redirects to `?next=` or the role home.

7 • Rider surface (`/rider`)

The rider surface is intentionally browsable without an account so visitors can price-quote a trip. Booking, however, requires sign-in. Layout: `app/rider/layout.tsx`; sub-routes that need a session enforce their own.

`/rider` • Hailing screen

`components/rider/rider-home.tsx`

- Greeting + live geolocation pill (detecting / city / off / unavailable).
- Full-bleed ambient map (CartoDB Dark Matter / Voyager) that flies to the user's GPS coordinates as soon as the location permission is granted.
- Quick-chip rail with the rider's saved Home and recent destinations.
- Mode toggle: **Driver** / **Package**.
- `LocationStack`: pickup + destination autocomplete (Nominatim via `/api/geocode`).
- Tier carousel rendered from `ride_tiers`; fares filled in by `/api/quote`.
- Primary CTA: *Request Ride* (signed-in) or *Sign in to request ride* (guest — persists pending form and round-trips through login).

`/rider/ride/[id]` · Active trip

`components/rider/rider-ride-view.tsx`

Single page that morphs with `ride.status`. Realtime subscription on `rides` + the driver's live location channel.

Status	Panel
<code>requested</code>	<code>WaitingForMatchPanel</code> — searching, with Cancel.
<code>accepted</code>	<code>DriverArrivingPanel</code> — driver card, ETA, plate badge, chat link.
<code>en_route_to_pickup</code> / <code>in_progress</code>	<code>StandardRidePanel</code> — full driver info, safety bar, chat.
<code>completed</code>	<code>CompletedRidePanel</code> — payment intent kicked off, 1–5 star rating.
<code>cancelled</code>	<code>CancelledRidePanel</code> — reason + back home.

`/rider/ride/[id]/chat` · In-ride chat

`app/rider/ride/[id]/chat/page.tsx`

Realtime message thread keyed by ride, backed by the `messages` table. Same component is reused on the driver side.

`/rider/history` · Activity

`app/rider/history/page.tsx`

Last 50 rides with stats card on top (trips, miles travelled, rating). Each completed-ride row has a star button to **favourite the driver**.

`/rider/profile` · Profile

`app/rider/profile/page.tsx`

Stats card, `ProfileForm` (avatar upload, name, phone, home location, delete account), favourite-drivers list, shortcut to history, and a prominent **Log out** button.

`/rider/wallet` & `/rider/ride-options` `app/rider/wallet/` · `app/rider/ride-options/`

Auxiliary pages — saved payment methods placeholder and a full grid view of available ride tiers (links from the home carousel's *View all*).

8 • Driver surface (/driver)

Gated by `requireRole("driver", "admin")` . Middleware additionally forces unapproved drivers through the application flow before any other driver page is accessible.

`/driver/apply` • Application wizard

`components/driver/application-wizard.tsx`

Multi-step submission form (personal details, national ID, license, vehicle, photos). Validates against `lib/driver/application-schema.ts` . Files upload to private Storage via signed URLs from `/api/driver/documents/upload-url` . Submission flips `drivers.approval_status` to `submitted` .

`/driver/onboarding` • Holding page

`app/driver/onboarding/page.tsx`

Shown after submission while an admin reviews. Polls `/api/driver/application/status` .

`/driver` • Dashboard

`components/driver/driver-home.tsx`

- Status card with name + rating + vehicle.
- Go online / Online toggle — upserts `drivers.status` , starts `useDriverLocationPublisher` (pushes GPS every `NEXT_PUBLIC_DRIVER_LOCATION_INTERVAL_MS`), and subscribes to a Realtime channel for incoming requests matching the driver's tier.
- Resume-active-trip banner.
- Incoming request cards with **Accept** (atomic RPC) / **Decline** (local dismiss).
- Daily earnings strip.

`/driver/ride/[id]` • Active trip

`components/driver/driver-ride-view.tsx`

The driver counterpart of the rider trip screen. The bottom panel exposes the state-machine button:

- `accepted` → *I'm at the pickup* → `en_route_to_pickup` .
- `en_route_to_pickup` → *Start trip* → `in_progress` .
- `in_progress` → *End trip* → `completed` . The server recomputes the fare from actual distance & duration and flips the driver back to `online` .

`/driver/earnings`

`app/driver/earnings/page.tsx`

Lifetime + recent-trip totals served by `/api/driver/earnings` .

`/driver/wallet`

`app/driver/wallet/page.tsx`

Driver-side wallet — payouts and balance placeholder.

`/driver/profile`

`app/driver/profile/page.tsx`

Stats card (trips, miles, rating, lifetime earnings), `ProfileForm`, `VehicleForm` (make, model, plate, colour, seats, requested tier), and a **Log out** button.

9 • Admin surface (`/admin`)

Gated by `requireRole("admin")`. `Layout ( app/admin/layout.tsx )` renders a persistent left sidebar on desktop and a slide-in drawer on mobile, plus a header with theme toggle, notifications bell, and the admin's profile pill. Every sub-route has an instant `loading.tsx` skeleton so navigation feels snappy.

`/admin` • Overview

`app/admin/page.tsx`

KPI tiles (riders, online drivers, active rides, revenue), live fleet map of every online driver (`AdminLiveMap` over Supabase Realtime), revenue mini-chart, latest rides table, top-driver leaderboard, and quick actions.

`/admin/users`

`app/admin/users/page.tsx`

Every profile with avatar, phone, trips, rating, and a three-pill role control (Rider / Driver / Admin). Promoting a rider to *driver* also upserts a `drivers` row so the driver portal works immediately. Admins cannot change their own role from this widget.

`/admin/drivers` & `/admin/drivers/[id]`

`app/admin/drivers/`

List of all drivers (status, vehicle, last-seen, approval status). The detail page shows the full application, document gallery (rendered from signed Storage URLs), and moderation actions (*approve / reject / warn / ban / unban / set tier / set rating / verify*). Every action is logged to `driver_moderation_log`.

`/admin/rides`

`app/admin/rides/page.tsx`

Last 200 rides with status-pill filters. Each row shows time, from, to, status, and fare.

/admin/pricing

components/admin/pricing-editor.tsx

Live editor for `ride_tiers` : base fare, per-km, per-minute, min fare, surge, currency, seat count, active flag. Saving here immediately changes what every rider sees on the home carousel and what `/api/quote` returns.

/admin/profile

app/admin/profile/page.tsx

Stats (users, trips, miles, revenue), the same `ProfileForm` as the other surfaces, and **Log out**.

10 · Backend API endpoints (/api/*)

Method	Route	Purpose
GET	/api/geocode	Forward geocoding (Nominatim proxy) for the autocomplete inputs.
GET	/api/geocode/reverse	Reverse geocoding for "use my current location" auto-fill.
POST	/api/quote	OSRM distance + duration, then runs every active tier through <code>lib/pricing/fare.ts</code> .
POST	/api/rides	Rider creates a ride. Inserts a row in status <code>requested</code> .
GET	/api/rides/[id]	Fetch a single ride (RLS gates visibility).
PATCH	/api/rides/[id]	State machine: <code>accept</code> (atomic RPC), <code>arrive</code> , <code>start</code> , <code>complete</code> , <code>cancel</code> . <code>complete</code> recomputes the fare from actual distance & duration.
POST	/api/rides/[id]/rate	Rider or driver submits a 1–5 star rating + optional comment.
POST	/api/payments/intent	Creates a payment intent through the active <code>PaymentProvider</code> (Stripe or mock).
POST	/api/payments/webhook	Stripe webhook receiver. Updates the <code>payments</code> row to <code>paid</code> / <code>failed</code> / <code>refunded</code> .
POST	/api/complaints	Rider or driver files a complaint about the other party in a ride.
POST	/api/driver/application	Submit / re-submit the driver application. Flips approval status to <code>submitted</code> .
GET	/api/driver/application/status	Lightweight polling endpoint used by the onboarding screen.
POST	/api/driver/documents/upload-url	Issues short-lived signed Storage URLs so the wizard uploads documents directly to Supabase without going through Next.js.
GET	/api/driver/earnings	Driver earnings summary (totals + per-trip breakdown).
PATCH	/api/admin/drivers/[id]	Admin moderation endpoint: approve, reject, warn, ban, unban, set tier, set rating, verify. Every call writes a row in <code>driver_moderation_log</code> .

Method	Route	Purpose
DELETE	<code>/api/profile</code>	Permanently delete the caller's account via service-role <code>auth.admin.deleteUser</code> . Cascades remove every downstream row.

11 · Shared libraries (lib/)

lib/auth/

- `routing.ts` — `isPublicPath`, `resolveHomePath`, `canAccess`.
- `session.ts` — `requireUser()` + `requireRole(...)` for server components.

lib/supabase/

- `client.ts` — browser Supabase client (anon key only).
- `server.ts` — request-bound server client + a service-role client used only by trusted route handlers (account deletion, payments webhook).
- `middleware.ts` — refreshes the session cookie on every request.
- `types.ts` — hand-maintained TypeScript types mirroring the schema.

lib/maps/

- `osrm.ts` — `routeBetween(a, b)` returns `{distanceKm, durationMin, polyline, coordinates}` with a haversine fallback.
- `nominatim.ts` — forward + reverse geocoding with a project-specific User-Agent.
- `types.ts` — `LatLng`, `RouteResult`, `PlaceSuggestion`.
- `use-geolocation.ts` — React hook wrapping `navigator.geolocation` as a state machine.

lib/pricing/

- `fare.ts` — `quoteFare({tier, distanceKm, durationMin})`. Pure function; unit-tested.
- `fare.test.ts` — Vitest coverage for the quote engine.

lib/payments/

- `provider.ts` — `PaymentProvider` TypeScript interface (`createIntent`, `verifyWebhook`, etc.).
- `stripe.ts` / `mock.ts` — concrete implementations.
- `index.ts` — factory selected by `PAYMENT_PROVIDER`.

lib/realtime/

- `channels.ts` — channel name conventions.
- `use-driver-location.ts` — `useDriverLocationPublisher` (driver side) + `useDriverLocation` (rider side).

- `use-ride.ts` — subscribe to a single ride's status updates.

lib/notifications/

- `admin.ts` — `buildAdminNotifications` (latest rides + new sign-ups, linked into `/admin/rides` & `/admin/users`).
- `user.ts` — `buildUserNotifications` (rider/driver feed of ride status events).

lib/admin/

- `driver-moderation.ts` — service-role helper for the moderation endpoint. Applies the action and writes a log row.
- `signed-storage-url.ts` — issues time-bound URLs so the document gallery can render private files.

lib/driver/application-schema.ts

Zod schema describing every wizard step + validation rules; reused server-side when accepting submissions.

lib/utils/

- `cn.ts` — tailwind class-name joiner.
- `format.ts` — `formatMoney` (zero-decimal currencies handled), `formatDistance`, `formatDuration`.

12 · Realtime, maps & geolocation

Realtime channels

- **Driver location** — the driver client publishes its GPS every `NEXT_PUBLIC_DRIVER_LOCATION_INTERVAL_MS` (5 s default) while online. The rider's active-trip screen subscribes to the driver's channel so the marker moves smoothly.
- **Ride status** — the rider (and driver) subscribe to their ride row in `rides`; status transitions trigger panel swaps without a reload.
- **Admin fleet** — `AdminLiveMap` subscribes to every `drivers` row and plots online drivers in realtime.
- **In-ride chat** — `messages` table is in the Realtime publication; new rows appear instantly on both sides.

Maps

- **Basemap** — CartoDB Dark Matter (dark theme) or Voyager (light theme). Theme-aware tile URL inside `RideMap`.
- **Markers** — custom div-icons drawn in the brand neon-green. Pickup pin, destination pin, animated car icon for the assigned driver, and small fleet dots for the admin view.
- **Route line** — three concentric strokes for a glow effect over the encoded polyline returned by OSRM.
- **Recentering** — `RideMap` exposes `recenterTo` (smooth `flyTo` without rendering a marker) and `markersHidden` (background-only use). The rider home uses both to follow the user's GPS coordinates.

Geolocation

- `useGeolocation` hook prompts for browser permission once on the rider home.
- On success, the page reverse-geocodes the fix through `/api/geocode/reverse` and auto-fills the Pickup field with the resolved address (falling back to raw coordinates if the lookup misses).
- An error boundary wraps the rider home so any failure surfaces a friendly retry card rather than crashing the page.

13 · Payments

Payments are abstracted so we can swap providers without touching the UI.

- `PaymentProvider` interface declares `createIntent`, `verifyWebhook`, etc.
- **Stripe** is the production implementation (Payment Intents API).
- **Mock** auto-completes intents for local development and demos. Selected when `PAYMENT_PROVIDER=mock`.
- The rider's completed-ride panel calls `POST /api/payments/intent` as soon as the ride hits `completed`.

- The Stripe webhook (`POST /api/payments/webhook`) updates `payments.status` to `paid` , `failed` , or `refunded` .

Production note. Stripe supports MWK but merchant onboarding in Malawi is limited. The `PaymentProvider` interface is designed so a local provider (e.g. PayChangu, Airtel Money) can be added by implementing one file.

14 · Environment configuration

Variables (names only — secrets are stored in the Render dashboard and never committed). See `.env.example` in the repo for shape.

Variable	Exposure	Purpose
<code>NEXT_PUBLIC_SITE_URL</code>	Public	Origin used for callbacks & webhooks.
<code>NEXT_PUBLIC_SUPABASE_URL</code>	Public	Supabase project URL.
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Public	Anon key, gated by RLS.
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Server only	Used by trusted route handlers (account deletion, payments webhook, moderation log inserts).
<code>OSRM_BASE_URL</code>	Server	Routing endpoint. Public demo is rate-limited — self-host for production.
<code>NOMINATIM_BASE_URL</code>	Server	Geocoding endpoint.
<code>PAYMENT_PROVIDER</code>	Server	<code>stripe</code> or <code>mock</code> .
<code>STRIPE_SECRET_KEY</code>	Server only	Stripe secret.
<code>NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY</code>	Public	Stripe publishable key.
<code>STRIPE_WEBHOOK_SECRET</code>	Server only	Webhook signature secret.
<code>NEXT_PUBLIC_DEFAULT_CURRENCY</code>	Public	Default ISO currency (MWK).
<code>NEXT_PUBLIC_DEFAULT_LAT</code> / <code>_LNG</code>	Public	Map fallback centre when geolocation is unavailable.
<code>NEXT_PUBLIC_DRIVER_LOCATION_INTERVAL_MS</code>	Public	Driver GPS publish cadence.
<code>RIDE_MATCH_RADIUS_KM</code>	Server	Max distance for driver search.

Secret values live in Render > Environment. Local development uses `.env.local` which is gitignored.

15 · Development & deployment workflow

Local scripts

Command	What it does
<code>npm run dev</code>	Next.js dev server on <code>localhost:3000</code> .
<code>npm run build</code>	Production build (used by Render).
<code>npm run start</code>	Run the production build locally.
<code>npm run lint</code>	ESLint.
<code>npm run typecheck</code>	<code>tsc --noEmit</code> .
<code>npm test</code>	Vitest (pricing engine).

Database migrations

Apply migrations in order from `supabase/migrations/` using the Supabase SQL editor (or the CLI's `db push`). Migrations are idempotent.

The seed file (`supabase/seed.sql`) inserts demo ride tiers and three test accounts (rider / driver / admin).

Branching & PRs

- Feature branches off `main`, prefixed `cursor/<short-name>-d15d` for agent-driven work.
- Pull requests target `main`; `main` auto-deploys to Render.
- Every commit must pass `npm run typecheck` and `npm run build`.

Hosting

- **Web** — Render web service (Node + Next.js). Auto-deploys on push to `main`.
- **Backend** — Supabase project. Schema kept in sync via the migrations folder.
- **Maps / geocoding** — currently the public OSRM & Nominatim instances; flip the env vars to a self-hosted or paid endpoint for production traffic.

16 · Glossary

Surface	A role-scoped section of the app — <i>rider</i> , <i>driver</i> , or <i>admin</i> . Each surface has its own layout, navigation, and Tailwind tokens.
Tier	A pricing tier in <code>ride_tiers</code> (e.g. Standard / Comfort / Lux). Drives both rider-side fare quotes and driver-matching.
RLS	Row-Level Security — Postgres policies enforced on every query max with the anon key.

RPC	A Postgres function callable via Supabase. We use <code>SECURITY DEFINER</code> functions for atomic operations (e.g. <code>accept_ride</code>) and admin predicates (<code>is_admin</code>).
Surge	Multiplier applied on top of the base fare formula. Configurable per tier and globally via <code>app_config</code> .
Approval status	State of a driver application — <code>draft</code> → <code>submitted</code> → <code>approved</code> / <code>rejected</code> / <code>banned</code> . Middleware blocks unapproved drivers from the dashboard.
Moderation log	Append-only audit trail of every admin action against a driver (approve, reject, warn, ban, unban, change tier, override rating, verify).

Document generated 2026-05-21 from the live `main` branch. Re-generate any time the architecture changes substantially.